# OOPSIE: Evaluating QLoRA Adaptation for Schema-Compliant Operational Failure Diagnostics

Onyx Prasai
onyxprasai@gmail.com

November 2026

**Abstract**

Deploying instruction-tuned language models as middleware in automated software diagnostics is frequently hindered by structural output instability under zero-tolerance parsing. We present OOPSIE (Operational Output Parser & Structured Inference Engine), an open evaluation framework examining whether Quantized Low-Rank Adaptation (QLoRA) improves structured JSON schema compliance for operational diagnostic reporting, and whether such adaptation alters qualitative diagnostic performance. We trained a QLoRA adapter (r=16, alpha=32) on the public model `google/gemma-4-12B-it` using a multi-turn error-recovery corpus and 4-bit NormalFloat quantization.

In a pre-specified frozen benchmark of 40 held-out failure scenarios (160 greedy-decoding passes), QLoRA adaptation improved strict JSON schema compliance from 0.0% in unadapted base-model conditions to 95.0% (38/40) with schema guidance and 97.5% (39/40) without schema guidance. Comparing unadapted zero-shot base inference to adapted inference with schema guidance was statistically significant under an exact two-sided McNemar test (38 discordant pairs; b=0, c=38; p approximately 7.28e-12; Edwards-corrected chi-squared=36.03, p approximately 1.95e-9). This result concerns programmatic schema compliance, not diagnostic correctness.

An author-adopted qualitative evaluation across all 160 completions, including blinded review of 40 responses, did not demonstrate a diagnostic-reasoning or technical-correctness advantage for adapted conditions over unadapted base models; no inter-rater reliability was computed. Within evaluated configurations, QLoRA adaptation improved structural output compliance. Constrained-decoding baselines were not evaluated; future work should compare adaptation with grammar-constrained decoding, JSON modes, and prompt-only controls. Artifacts and result archives are publicly available.

**Keywords:** Large Language Models, Parameter-Efficient Fine-Tuning, QLoRA, JSON Schema Conformance, Automated Program Repair, Operational Diagnostics.

## 1 Introduction

Modern instruction-tuned foundation language models possess extensive parametric knowledge regarding software engineering and debugging, as demonstrated on code generation benchmarks such as HumanEval [CTJ+21] and repository-level issue resolution benchmarks such as SWE-bench [JYW+24]. However, deploying general-purpose chat models as programmatic middleware in automated toolchains presents severe operational challenges. Under standard greedy decoding, unadapted chat models frequently emit conversational pleasantries (*"Certainly, here is the explanation..."*), enclose JSON payloads in Markdown code fences (```` ```json ... ``` ````), or introduce syntactic defects such as illegal escape sequences. When downstream tooling enforces zero-tolerance RFC 8259 JSON validation [Bra17] and rigid schema conformance (JSON Schema Draft 2020-12; [WAHD22, WAH22]), these formatting deviations cause immediate automated ingestion failure.

Prior work has addressed output structuring primarily through decoding-time grammar constraints, such as finite-state automata (FSM) guidance [WL23] or context-free grammar (CFG) token masking [USK+25]. While decoding-time masking guarantees syntax correctness, it imposes non-trivial serving requirements, custom runtime dependencies, and token-level vocabulary masking overhead. An alternative approach is parameter-efficient fine-tuning (PEFT), such as Low-Rank Adaptation (LoRA; [HSW+22]) and Quantized Low-Rank Adaptation (QLoRA; [DPHZ23]), which internalizes formatting and task conventions directly into the model's weights.

In this work, we present **OOPSIE** (Operational Output Parser & Structured Inference Engine), an open evaluation framework designed to address the following research question:

> *Does QLoRA adaptation improve structured JSON/schema compliance for operational-failure diagnostic reporting, and does it improve qualitative diagnostic reasoning?*

We evaluate this question on a pre-specified, frozen benchmark comprising 40 diverse, held-out software and infrastructure failure scenarios (160 total model completions across four experimental conditions). Our empirical results demonstrate that QLoRA adaptation dramatically improves programmatic schema compliance from 0.0% to 95.0%–97.5% under zero-tolerance greedy parsing ($p \approx 7.28 \times 10^{-12}$). However, a systematic qualitative evaluation reveals that domain adaptation confers **no observable advantage in diagnostic reasoning or technical correctness** over the unadapted base model.

## 2 Related Work

### 2.1 Structured Generation in Foundation Models

Enforcing structured outputs from autoregressive language models is critical for tool use and workflow orchestration. Constrained decoding approaches such as Outlines [WL23] index regular expressions or JSON schemas into deterministic finite automata (DFAs) that mask invalid next-token candidates at each generation step. SynCode [USK+25] generalizes this to context-free grammars (CFGs). In educational and developer error explanation tasks, Leinonen et al. [LHS+23] observed that while LLMs provide helpful diagnostic hints, unadapted models frequently introduce conversational formatting and preamble artifacts that prevent automated programmatic ingestion.

### 2.2 Parameter-Efficient Fine-Tuning

Low-Rank Adaptation (LoRA; [HSW+22]) parameterizes weight updates via low-rank decomposition matrices ($W = W_0 + \frac{\alpha}{r}BA$). QLoRA [DPHZ23] combines 4-bit NormalFloat (NF4) base quantization with double quantization and compute-type casting, allowing 12B+ parameter models to be adapted on single commodity GPUs. While PEFT has been evaluated on code generation and general NLP benchmarks [RGG+23,LAZ+23], its efficacy at inducing rigid multi-key JSON schemas under zero-tolerance parsing without grammar masking remains sparsely benchmarked.

### 2.3 Automated Program Repair and Fault Diagnosis

Automated Program Repair (APR) traditionally encompasses fault localization, patch synthesis, and patch validation against test suites [ZFM+24]. As surveyed by Hou et al. [HZL+24], foundation models excel at synthesizing candidate patches but risk benchmark leakage when evaluated on popular public repositories. OOPSIE is designed not as an autonomous repair agent, but as an advisory operational diagnostic parser that translates unstructured system logs [KS06] into strictly validated machine-readable incident structures.

# 3 Method

## 3.1 Canonical Diagnostic Schema

OOPSIE targets a standardized 10-key JSON schema compliant with JSON Schema Draft 2020-12 [WAHD22, WAH22]. Every completion must contain exactly the following top-level keys without additional properties (`additionalProperties: false`):

1. `schema_version`: Fixed string enum (`"1.0.0"`).
2. `error_category`: Closed vocabulary enum (10 valid operational categories).
3. `confidence`: Numeric float in the closed interval $[0.0, 1.0]$.
4. `risk_level`: Categorical enum (`"low"`, `"medium"`, `"high"`).
5. `summary`: Concise diagnostic string ($\geq 10$ characters).
6. `root_cause`: Deep technical failure attribution ($\geq 15$ characters).
7. `suggested_fix`: Actionable developer guidance ($\geq 15$ characters).
8. `safety_warning`: Risk mitigation advisory ($\geq 10$ characters).
9. `repair_prompt`: Direct, copy-ready prompt string for coding agents ($\geq 20$ characters).
10. `verification_steps`: Non-empty array of executable verification commands/checks ($\geq 1$ item).

## 3.2 Training Corpus and Data Governance

The training dataset was curated from 15 real-world open-source repositories across 10 functional software domains (web development, backend databases, cloud infrastructure, container orchestration, machine learning pipelines, etc.). Each scenario was mapped to an incident log and paired with an expert-verified 10-key JSON payload. The corpus was partitioned into 100 training examples, 20 validation examples, and 20 test examples. All 40 benchmark scenarios were strictly isolated and held out from all training and hyperparameter tuning.

## 3.3 Model Architecture and QLoRA Adaptation

We trained a QLoRA adapter ($r = 16, \alpha = 32$) on top of the public instruction-tuned foundation model `google/gemma-4-12B-it`[1] ( 11.95B dense parameters; [Gem26, Goo26]). The base model was loaded in 4-bit NormalFloat (NF4) with 16-bit bfloat compute dtype and double quantization. Trainable low-rank decomposition matrices were attached to all linear attention and feed-forward projections:

$$W = W_0 + \frac{\alpha}{r} BA, \quad r = 16, \quad \alpha = 32 \tag{1}$$

yielding 42.6 million trainable parameters (0.35% of total base model parameters). Training was executed for 2 epochs (44 optimization steps) using AdamW with cosine learning rate scheduling ($2 \times 10^{-4}$ peak), weight decay 0.01, and sequence length 2,048 tokens on an NVIDIA Tesla T4 GPU (16 GB VRAM). Checkpoint selection was determined by validation loss minimum on `data/validation.jsonl`.

[1] Provenance note: The official public Hugging Face repository identifier is `google/gemma-4-12B-it`. Historical training run configuration files (`run_config.json` and `adapter_config.json`) in this repository recorded the lowercase identifier `google/gemma-4-12b-it`; both identifiers resolve to the identical model checkpoint on Hugging Face.

# 4 Experimental Setup

## 4.1 Four Experimental Conditions

To rigorously isolate the effects of QLoRA adaptation versus prompt engineering, we evaluate four conditions across all 40 held-out benchmark scenarios:

- **Condition A (Base Zero-Shot + Schema)**: Unadapted base model with full 710-token JSON schema embedded in the system prompt.
- **Condition B (Adapted + Schema)**: QLoRA adapted model with full JSON schema in the system prompt.
- **Condition C (Base One-Shot + Schema)**: Unadapted base model with full JSON schema and one complete gold-standard input/output demonstration in context.
- **Condition D (Adapted No-Schema)**: QLoRA adapted model with minimal task prompt, completely omitting the schema text.

## 4.2 Decoding Parameters and Ingestion Pipeline

All 160 completions were generated using greedy decoding (`temperature = 0.0`, `top_p = 1.0`, `max_new_tokens = 1536`) to ensure deterministic reproducibility. Programmatic evaluation employs a zero-tolerance ingestion pipeline:

1. **Raw JSON Parse**: `json.loads(output)` without Markdown fence stripping or regex extraction.
2. **Lenient JSON Parse**: Extraction after stripping leading/trailing whitespace and Markdown code fences (```` ```json ... ``` ````).
3. **Strict Schema Validation**: Formal JSON Schema Draft 2020-12 validation against `data/schema.json`.
4. **Key Completeness**: Verification that all 10 canonical keys are present and no unexpected keys exist.

## 4.3 Blinded Qualitative Evaluation

To evaluate diagnostic reasoning without confirmation bias, completions from all four conditions across the first 10 benchmark scenarios (40 total completions) were assigned cryptographic tokens and blinded. An evaluator assessed each completion across five dimensions: Technical Correctness, Root Cause Accuracy, Fix Actionability, Safety Appropriateness, and Repair Prompt Utility. The remaining 120 completions were evaluated using an automated evaluation pipeline applying the frozen rubric, which were subsequently author-reviewed and adopted. **Of the 160 total responses: 40 were included in the blinded/human-review workflow; the remaining 120 relied on unreviewed AI-generated provisional rubric ratings selectively reviewed and adopted by the author. No Cohen's $\kappa$, Krippendorff's $\alpha$, or other inter-rater reliability statistic was computed. These qualitative results are provisional and exploratory and should not be treated as a validated human evaluation.**

| Condition | Raw JSON | Lenient JSON | Strict Schema | Key Compl. | Latency (s) | Fail Reason |
|---|---|---|---|---|---|---|
| **A: Base Zero-Shot + Schema** | 0/40 (0.0%) | 40/40 (100.0%) | 0/40 (0.0%) | 0/40 (0.0%) | $118.62 \pm 12.73$ | Markdown fences / preambles |
| **B: Adapted + Schema** | 38/40 (95.0%) | 38/40 (95.0%) | 38/40 (95.0%) | 38/40 (95.0%) | $114.42 \pm 11.53$ | Max tokens cutoff (2 runs) |
| **C: Base One-Shot + Schema** | 0/40 (0.0%) | 40/40 (100.0%) | 0/40 (0.0%) | 0/40 (0.0%) | $119.35 \pm 9.79$ | Markdown fences / preambles |
| **D: Adapted No-Schema** | 40/40 (100.0%) | 40/40 (100.0%) | 39/40 (97.5%) | 39/40 (97.5%) | $99.88 \pm 11.93$ | Missing 1 key (1 run) |

Table 1: Programmatic parsing and schema compliance results across 40 held-out scenarios (160 total passes). Latency denotes single sequential execution time on an NVIDIA Tesla T4 GPU (mean ± standard deviation).

### 4.4 Relationship to Constrained Decoding and JSON Mode

This study did not evaluate constrained-decoding baselines, provider JSON modes, or grammar-guided decoding. Therefore, the results do not establish that QLoRA adaptation is necessary or preferable to constrained decoding for achieving schema compliance. Specifically, the following approaches were not evaluated:

- **Grammar-constrained decoding** (e.g., FSM/CFG masking as in Outlines [WL23] or SynCode [USK+25]).
- **Provider JSON modes** (inference-time JSON enforcement offered by hosted model providers).
- **Negative-prompting baselines** (e.g., system-prompt instructions such as *"Output raw JSON only, no markdown, no preamble"*).

The evaluation scope was restricted to parameter adaptation versus standard prompt-only baselines using identical inference infrastructure, without integration of constrained-decoding runtime engines. Future work should compare QLoRA adaptation against constrained-decoding approaches, JSON-mode decoding, and prompt-only formatting controls to determine whether adaptation is necessary or merely one of several viable paths to structural compliance.

## 5 Results

### 5.1 Programmatic Parsing and Schema Compliance

Table 1 reports the quantitative parsing and schema conformance results across all 160 runs.

Unadapted base models (Conditions A and C) achieved 0.0% raw JSON parsing and 0.0% strict schema compliance. In 100% of cases, failure resulted from enclosing the output in Markdown code blocks or including conversational introductory text. Once fences were stripped (Lenient JSON), the base models proved to have generated valid JSON syntax (100.0%).

Conversely, QLoRA adaptation successfully internalized direct JSON emission: Condition B achieved 95.0% (38/40) raw and strict schema compliance, while Condition D achieved 100.0% raw parsing and 97.5% (39/40) strict schema compliance. In Condition B, the two non-compliant completions were truncated due to the 1,536-token generation limit. In Condition D, the single non-compliant completion omitted one required key (`risk_level`) while successfully producing the other 9 keys.

### 5.2 Inferential Statistical Significance

To rigorously test whether the observed improvement in schema compliance is statistically significant, we construct the paired $2 \times 2$ contingency table across the 40 scenarios for Condition A vs. Condition B, shown in Table 2.

| | Condition B Pass | Condition B Fail |
|---|---|---|
| **Condition A Pass** | 0 ($a$) | 0 ($b$) |
| **Condition A Fail** | 38 ($c$) | 2 ($d$) |

Table 2: Paired $2 \times 2$ contingency table for strict schema compliance (Condition A vs. Condition B, $N = 40$ scenarios).

| Dimension | Condition A (Base) | Condition B (Adapted) | Condition C (1-Shot) | Condition D (No-Schema) |
|---|---|---|---|---|
| **Technical Correctness** | 37 Pass / 3 Partial / 0 Fail | 37 Pass / 3 Partial / 0 Fail | 40 Pass / 0 Partial / 0 Fail | 37 Pass / 3 Partial / 0 Fail |
| **Root Cause Accuracy** | 38 Pass / 2 Partial / 0 Fail | 38 Pass / 2 Partial / 0 Fail | 40 Pass / 0 Partial / 0 Fail | 38 Pass / 2 Partial / 0 Fail |
| **Fix Actionability** | 38 Pass / 2 Partial / 0 Fail | 38 Pass / 2 Partial / 0 Fail | 40 Pass / 0 Partial / 0 Fail | 38 Pass / 2 Partial / 0 Fail |
| **Safety Appropriateness** | 40 Pass / 0 Partial / 0 Fail | 38 Pass / 2 Partial / 0 Fail | 40 Pass / 0 Partial / 0 Fail | 38 Pass / 2 Partial / 0 Fail |
| **Repair Prompt Utility** | 39 Pass / 1 Partial / 0 Fail | 38 Pass / 2 Partial / 0 Fail | 40 Pass / 0 Partial / 0 Fail | 37 Pass / 3 Partial / 0 Fail |

Table 3: Systematic qualitative evaluation across all 160 completions (40 scenarios × 4 conditions). *Rating provenance: 40/160 responses were included in the blinded/human-review workflow; 120/160 relied on unreviewed AI-generated provisional ratings selectively reviewed and adopted by the author. No inter-rater reliability statistic was computed.*

The contingency table yields 38 discordant pairs ($b = 0$, $c = 38$). Under the exact two-sided McNemar test with null hypothesis $H_0 : P(b) = P(c) = 0.5$:

$$p = 2 \times \sum_{k=0}^{0} \binom{38}{k} (0.5)^{38} = 2 \times (0.5)^{38} \approx 7.276 \times 10^{-12} \tag{2}$$

The Edwards continuity-corrected chi-square statistic is:

$$\chi^2_{\text{corrected}} = \frac{(|b - c| - 1)^2}{b + c} = \frac{(|0 - 38| - 1)^2}{38} = \frac{37^2}{38} = \frac{1369}{38} \approx 36.0263 \tag{3}$$

yielding $p \approx 1.947 \times 10^{-9}$ ($df = 1$). The uncorrected McNemar statistic $\chi^2 = 38.00$ yields $p \approx 7.074 \times 10^{-10}$.

*Statistical Scope and Limitations*: We explicitly specify that this significance claim pertains strictly to the primary outcome of programmatic schema compliance under zero-tolerance greedy parsing on the 40 paired benchmark scenarios. The p-values are unadjusted for multiple hypothesis testing, and the test does not measure, imply, or support any claim of superiority in underlying technical diagnostic correctness.

### 5.3 Inference Latency Profile

Omitting the explicit schema text from the prompt (Condition D) yielded an observed mean single-run latency reduction of 14.54 seconds (−12.7%) compared to Condition B (99.88±11.93 s vs. 114.42 ± 11.53 s). We note that latency was measured during a single sequential run on an isolated NVIDIA Tesla T4 GPU with batch size 1 and unpadded inputs; this latency profile is reported as an operational observation of prompt-length savings rather than a general hardware benchmark guarantee.

### 5.4 Qualitative Diagnostic Reasoning Review

Table 3 summarizes the qualitative evaluation across all 160 completions.

Remarkably, qualitative diagnostic performance was virtually identical between unadapted and adapted models:

- Across all 160 completions, zero completions received a **Fail**, **Severe Risk**, or **Cannot Assess** rating.
- Condition C (Base 1-Shot) achieved 40/40 Pass on technical correctness and root cause accuracy, whereas Conditions B and D each achieved 37/40 Pass and 3/40 Partial.
- Error attribution errors in the Partial cases (e.g., misattributing an expired SSL certificate to an incorrect intermediate authority) occurred with equivalent frequency across base and adapted completions.

# 6 Discussion

## 6.1 Parameter Adaptation as a Structural Filter

The central empirical finding of this work is that QLoRA adaptation functioned primarily as a structural output-format intervention rather than an amplifier of diagnostic reasoning. The base model `google/gemma-4-12B-it` already possesses comprehensive parametric domain knowledge regarding software faults, compilation errors, and distributed systems exceptions. However, its default conversational instruction tuning induces Markdown fences and conversational wrappers that break automated ingestion. Within the evaluated configurations, QLoRA adaptation improved structural output compliance substantially, aligning the generation distribution to output bare RFC 8259 JSON without decoding-time grammar constraints.

As noted in Section 4.4, constrained-decoding baselines, provider JSON modes, and negative-prompting controls were not evaluated. The results therefore do not establish that QLoRA adaptation is necessary or preferable to these alternatives. Future work should directly compare parameter adaptation against constrained-decoding and prompt-only formatting controls.

## 6.2 Prompt Compaction and Schema Omission

Condition D demonstrates that the adapted model internalizes the 10-key schema structure so completely that the prompt schema can be omitted entirely, reducing prompt context length by 710 tokens and cutting end-to-end execution latency by 12.7% without degrading schema compliance (97.5%).

## 6.3 Operational Safety Implications

Because parameter adaptation does not enhance underlying reasoning, it must not be treated as a substitute for validation. Organizations adopting PEFT models for automated operational pipelines must implement strict validation boundaries:

1. **Syntactic and Schema Gate**: Automatic validation against the formal JSON schema before downstream consumption.
2. **Human-in-the-Loop Gate**: Requiring explicit developer review before executing suggested fixes or repair prompts in production environments.

# 7 Limitations

We explicitly document the threats to validity and limitations of this study:

1. **Sample Size**: The benchmark contains 40 curated scenarios. While statistically significant for structural parsing ($p < 10^{-11}$), larger corpora are needed to detect subtle qualitative shifts.

2. **Evaluator Scope**: 40 completions were evaluated under a blinded protocol, while the remaining 120 completions were author-adopted AI-assisted evaluations. These do not constitute independent multi-rater inter-coder reliability metrics.

3. **Synthetic Contexts**: Scenarios were curated from real open-source repositories but evaluated as static isolated logs rather than live interactive environments.

4. **Model Specificity**: Findings are specific to `google/gemma-4-12B-it` and the OOPSIE 10-key schema; generalization to other foundation models or schemas requires empirical verification.

5. **No Constrained-Decoding or JSON-Mode Baselines**: This study did not evaluate grammar-constrained decoding, provider JSON modes, FSM/CFG token masking, or negative-prompting controls. The results do not establish whether QLoRA adaptation is necessary or preferable to these alternatives for achieving schema compliance.

## 8 Ethics and Safety

Automating system error remediation presents inherent operational risks. Flawed diagnostic suggestions could lead to catastrophic downtime, data loss, or security vulnerabilities (e.g., executing destructive shell commands or disabling security controls). OOPSIE outputs are strictly advisory; all repair prompts and suggested fixes must be validated by human engineers.

## 9 Reproducibility

All benchmark datasets, evaluation protocols, schemas, and analysis scripts are publicly available in the OOPSIE repository at https://github.com/Onyx-Prasai/OOPSIE. The frozen evaluation bundle and benchmark result archive are published under GitHub Release v1.2.0:

- Evaluation Bundle ZIP SHA-256:
  `259bd473d9d18e0b13b79591ca1e44b0439667131b9e3966468d044ca05e42c5`
- Public Result Archive SHA-256:
  `cf24eec1a5c3347ae826fec4b5cbfb7133a1c4ce92aace3eab2b048bdd37f4eb`

## 10 Conclusion

This study investigated whether parameter-efficient fine-tuning via QLoRA can improve structural JSON schema compliance for operational software error recovery, and whether it alters underlying diagnostic reasoning. In a pre-specified frozen benchmark across 40 unseen failure scenarios (160 total passes), QLoRA adaptation of `google/gemma-4-12B-it` dramatically improved programmatic schema compliance from 0.0% to 95.0%–97.5% under zero-tolerance greedy parsing ($p \approx 7.28 \times 10^{-12}$), eliminating conversational preambles and Markdown code fences. Crucially, the qualitative evaluation did not identify a diagnostic-reasoning or technical-correctness advantage for the adapted conditions over the unadapted base-model conditions. Within this benchmark, QLoRA adaptation functioned primarily as a structural output-format intervention rather than an amplifier of diagnostic reasoning. Constrained-decoding baselines were not evaluated; future work should compare adaptation against grammar-constrained decoding, JSON-mode decoding, and prompt-only formatting controls. Consequently, operational deployments can leverage QLoRA for reliable automated JSON ingestion, but must maintain rigorous validation boundaries and human developer oversight.

## AI Use Disclosure

The author used AI-assisted tools during research and manuscript preparation. Initial qualitative ratings for the 160 benchmark completions were generated by an automated evaluation pipeline using a frozen rubric. The author reviewed key outputs, verification findings, and flagged evaluations, and takes full responsibility for the final work. The remaining provisional qualitative ratings were retained without case-by-case author review and are reported as author-adopted AI-assisted evaluations. These ratings are not independent human inter-rater assessments. AI tools were also used to assist with code development, documentation, manuscript drafting, and consistency checking.

## References


[Bra17] Tim Bray. The JavaScript object notation (JSON) data interchange format. RFC 8259, Internet Engineering Task Force (IETF), December 2017.

[CTJ+21] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Dennison, Florian Sigler, Nikolas Prashanth, Shyamal Arora, Pranav Chantzis, Mateusz Gray, Kimberly Zwierzchowska, and Wojciech Zaremba. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

[DPHZ23] Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. QLoRA: Efficient finetuning of quantized LLMs. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, pages 10088–10115, 2023.

[Gem26] Gemma Team (Google DeepMind). Gemma 4 technical report. Preprint arXiv:2607.02770, arXiv, July 2026.

[Goo26] Google LLC. google/gemma-4-12b-it model card and repository. Hugging Face, 2026.

[HSW+22] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2022.

[HZL+24] Xinyi Hou, Yanjie Zhao, Yue Liu, Zhou Yang, Kailong Wang, Li Li, Xiapu Luo, David Lo, John Grundy, and Haoyu Wang. Large language models for software engineering: A systematic literature review. *ACM Transactions on Software Engineering and Methodology (TOSEM)*, 2024.

[JYW+24] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*, 2024.

[KS06] Karen Kent and Murugiah Souppaya. Guide to computer security log management. Special Publication (SP) 800-92, National Institute of Standards and Technology (NIST), September 2006.

[LAZ+23] Raymond Li, Loubna Ben Allal, Yangtian Zi, Niklas Muennighoff, Denis Kocetkov, Chenghao Mou, Marc Marone, Christopher Akiki, Jia Li, Jenny Chim, Qian Liu, Evgenii Zheltonozhskii, Terry Yue Zhuo, Thomas Wang, Olivier Dehaene, Mishig

Davaadorj, Joel Lamy-Poirier, João Monteiro, Oleh Shliazhko, Nicolas Patry, Anton Lozhkov, Mohammad Amini, Loubna Ben Allal, Leandro von Werra, and Harm de Vries. StarCoder: may the source be with you! *arXiv preprint arXiv:2305.06161*, 2023.

[LHS+23] Juho Leinonen, Arto Hellas, Sami Sarsa, Brent Reeves, Paul Denny, James Prather, and Brett A. Becker. Using large language models to enhance programming error messages. In *Proceedings of the 54th ACM Technical Symposium on Computer Science Education (SIGCSE)*, volume 1, pages 452–458, 2023.

[RGG+23] Baptiste Rozière, Jonas Gehring, Fabian Gloeckle, Sten Sootla, Itai Gat, Xiaoqing Ellen Tan, Yossi Adi, Jingyu Liu, Tal Remez, Jérémy Rapin, Artyom Kozhevnikov, Ivan Evtimov, Gabriel Synnaeve, and Louis Martin. Code llama: Open foundation models for code. *arXiv preprint arXiv:2308.12950*, 2023.

[USK+25] Shubham Ugare, Tarun Suresh, Hangoo Kang, Sasa Misailovic, and Gagandeep Singh. SynCode: LLM generation with grammar augmentation. *Transactions on Machine Learning Research (TMLR)*, 2025.

[WAH22] Austin Wright, Henry Andrews, and Ben Hutton. JSON schema validation: A vocabulary for structural validation of JSON. Internet-Draft draft-bhutton-json-schema-validation-01, Internet Engineering Task Force (IETF), June 2022. Draft 2020-12 dialect.

[WAHD22] Austin Wright, Henry Andrews, Ben Hutton, and Greg Dennis. JSON schema: A media type for describing JSON documents. Internet-Draft draft-bhutton-json-schema-01, Internet Engineering Task Force (IETF), June 2022. Draft 2020-12 dialect.

[WL23] Brandon T. Willard and Rémi Louf. Efficient guided generation for large language models. *arXiv preprint arXiv:2307.09702*, 2023.

[ZFM+24] Quanjun Zhang, Chunrong Fang, Yuxiang Ma, Weisong Sun, and Zhenyu Chen. A survey of learning-based automated program repair. *ACM Transactions on Software Engineering and Methodology (TOSEM)*, 33(2):39:1–39:45, 2024.